Open Source CFD

Tutorial #3: Orifice plate

Axisymmetrical Flow: Orifice Plate with simpleFoam

Robert Castilla
Department of Fluid Mechanics

2025-26
Version 1.0
Learning Objectives:

- Create a blockMesh dictionary with python
- Extrude the mesh to create a axisymmetrical simmulation
- Run a simple steady-state simulation
- Monitor residuals from terminal
- Analyze basic results in terminal and paraview
- Validate results with standard publication

This tutorial introduces the use of a python interface to create a blockMesh dictionary. The study case is the a simple orifice plate, which will be simulated with `simpleFoam`. It is not available in tutorials and it will be generated from a template case. Pressure loss and dimensionless coefficient will be obtained both with post-processing utilities in terminal and with paraview.

Contents

1 Introduction and Theoretical Background

1.1 What is an orifice plate?

An orifice plate is a device used to experimentally measure the flow rate in a pipe. It is basically a circular plate, with an orifice in the center, that restricts the flow and produce a pressure loss. This pressure loss is mathematically related to the flow rate in the pipe.

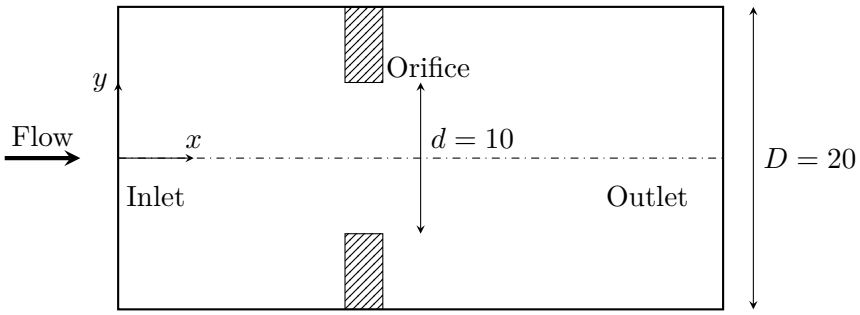

Figure 1: Orifice plate geometry section. Dimensions are in mm.

More information can be found on Wikipedia.

1.2 Flow rate measurement

In the orifice plate flow rate is computed with equation (1)

$$Q = C_d \frac{A_o}{\sqrt{1-\beta^4}} \sqrt{\frac{2\Delta p}{\rho}} \tag{1}$$

where $\beta = d/D$,A_o is the area of the orifice, ρ is the density of the fluid , Δp is the pressure loss in the orifice and C_d is the **coefficient of discharge**. The approximate value of Cd is 0.6, but it can be more precisely estimated with the Reader-Harris/Gallagher equation[1]

$$
\begin{aligned}
C_d =& 0.5961 + 0.0261\beta^2 - 0.216\beta^8 + 0.000521\left(\frac{10^6\beta}{Re_D}\right)^{0.7} \\
&+ (0.0188 + 0.0063A)\beta^{3.5}\left(\frac{10^6}{Re_D}\right)^{0.3} \\
&+ (0.043 + 0.080e^{-10L_1} - 0.123e^{-7L_1})(1 - 0.11A)\frac{\beta^4}{1-\beta^4} \\
&- 0.031(M_2' - 0.8M_2'^{1.1})\beta^{1.3} + 0.011(0.75 - \beta)\left(2.8 - \frac{D}{0.0254}\right)
\end{aligned} \tag{2}
$$

where

$$
\begin{aligned}
\beta &= d/D \quad \text{(diameter ratio)} \\
Re_D &= \text{Reynolds number based on pipe diameter} \\
A &= \left(\frac{19000\beta}{Re_D}\right)^{0.8} \\
M_2' &= \frac{2L_2'}{1-\beta}
\end{aligned}
$$

The value of Δp depends on where the pressure are measured upstream and downstream with relation to the orifice. In this case we are going to measure the pressure upstream (p_1) at a distance D and downstream (p_2) at a distance $\frac{D}{2}$. According to ISO 5167 standard[2], for this case $L_1' = 1$ and $L_2' = 0.47$.

2 Simulation setup

2.1 Case Overview

Parameter	**Value**
Solver	`simpleFoam` (steady, incompressible)
Turbulence model	k-ω - SST
Flow regime	Turbulent
Geometry	orifice plate
Reynolds number	2×10^4 (based on pipe diameter)
Expected runtime	60-80 seconds

Table 1: Orifice plate case specifications

2.2 Case copy

Since this case is not available in the tutorials folder, we have two options. We can either adapt a tutorial (for instance, the pitzDaily tutorial) or we can use a `template` case, that can be found in `$FOAM_ETC/templates`.

```
# load OpenFOAM if it is not included in the .bashrc file
source /usr/lib/openfoam/openfoam-2312/etc/bashrc

# Make a local folder
mkdir -p ~/Tutorials/Tutorial_3

# Copy the inflowOutflow in this folder
cp -r $FOAM_ETC/templates/inflowOutflow ~/Tutorials/Tutorial_3/orificePlate
# Note that we have changed the name with the copy

# Go to the tutorial place
cd ~/Tutorials/Tutorial_3/orificePlate
```

This case is a simple example of a fluid domain with an inlet and an outlet. In the `README` file you can read the instructions to use the template. It is intended to be used with a general 3D geometry meshed with **`snappyHexMesh`**. Since we are going to use **`blockMesh`** with an axisymmetrical mesh, this file can be removed

```
# remove the README file
rm README
```

2.3 Mesh generation with blockMesh and Python

The mesh is block-structured with 5 blocks, 2 upstream, 2 downstream and 1 in the orifice. Generating this mesh with **`blockMesh`** and a dictionary `system/blockMeshDict` by hand can be tedious and prone to errors. Instead, we are going to use the python package ofblockmeshdicthelper to generate the mesh with a python script. The main advantages of using a python code for the mesh generation are:

1. It is more **readable**. Vertexes, blocks, patches... have meaningful names instead of just coordinates and integers indexes.
2. It is a **parametric** design. The entire mesh can be regenerated with the modification of a geometric feature.
3. It is much easier to use python to perform **complex mathematical computations** for geometry, grading,...
4. It is possible to make an **automated process**. The python code can be adapted to be called with options in the command and it can be included, for instance, in a loop in a bash script, or in another python code. Possibilities are infinity...

For an axisymmetrical simulation, we define a **wedge** mesh, with and small angle (in this case it will be 1°) with respect to the XY plane.

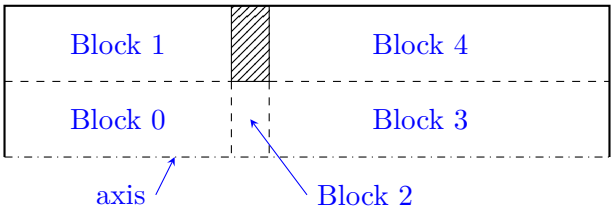

Figure 2: Blocks for the orifice plate mesh.

To install the helper, first activate the python environment.

```
# Activate the python evironment (see Tutorial#0, section 5.2)
source ~/my_env/bin/activate

# Install the package
pip install git+https://github.com/rclUPC/ofblockmeshdicthelper.git

# Clone the repository for examples
git clone https://github.com/rclUPC/ofblockmeshdicthelper.git
# Open the example notebook of the orifice plate
jupyter lab ofblockmeshdicthelper/examples/OFBlockMeshDictHelper_OrificePlate.
    ipynb &
```

Listing 1: Installing `ofblockmeshdicthelper` and opening the notebook for the orifice plate meshing

Remember that the final `&` is for passing the process to the background and have the terminal available for other commands.

The example note book will open in a `jupyter lab` session in the web browser

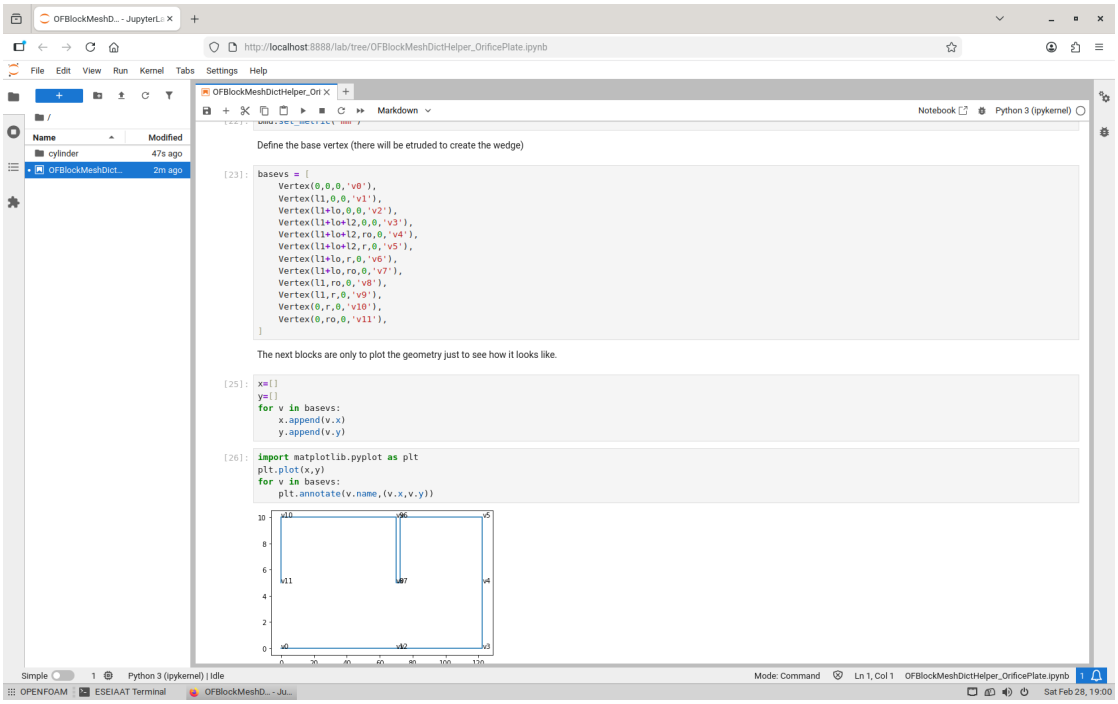

Figure 3: Screen shot of the jupyter lab session with the orifice plate meshing notebook

The jupyter notebook is quite self-explanatory. It can be executed cell by cell with **Shift-Enter** keys, but in this tutorial we are going to write a python script based on this jupyter notebook. It will make its execution faster and more practical.

Create a file `generate_OrificePlate_mesh.py` with the content of listing 2

```
#!/usr/bin/env python
# coding: utf-8

import numpy as np
from ofblockmeshdicthelper import *


# Define the geometry and the discretization

wedgeangle = 1
r = 10
ro = 5
l1 = 70
lo = 2.5
l2 = 50
deltax = 0.25
deltay = 0.25
Nx1 = int(l1/deltax)
```

```
Nx2 = int(lo/deltax)
Nx3 = int(l2/deltax)
Ny1 = int(ro/deltay)
Ny2 = int((r-ro)/deltay)

# Definition of the dictionary object

bmd = BlockMeshDict()


# Define the scale (geometry has been typed in mm)
md.set_metric('mm')


# Define the base vertex (there will be etruded to create the
    wedge)

basevs = [
Vertex(0,0,0,'v0'),
Vertex(l1,0,0,'v1'),
Vertex(l1+lo,0,0,'v2'),
Vertex(l1+lo+l2,0,0,'v3'),
Vertex(l1+lo+l2,ro,0,'v4'),
Vertex(l1+lo+l2,r,0,'v5'),
Vertex(l1+lo,r,0,'v6'),
Vertex(l1+lo,ro,0,'v7'),
Vertex(l1,ro,0,'v8'),
Vertex(l1,r,0,'v9'),
Vertex(0,r,0,'v10'),
Vertex(0,ro,0,'v11'),
]

# The extruded wedges are calculated and included in the
    dictionary

cosd = np.cos(np.radians(wedgeangle))
sind = np.sin(np.radians(wedgeangle))
for v in basevs:
bmd.add_vertex(v.x, v.y*cosd, v.y*sind, v.name+'+z')
bmd.add_vertex(v.x, v.y*cosd, -v.y*sind, v.name+'-z')


# Some of the vertexs are duplicated and then they can be "
    collapsed" in only one

bmd.reduce_vertex('v0-z','v0+z')
bmd.reduce_vertex('v1-z','v1+z')
bmd.reduce_vertex('v2-z','v2+z')
bmd.reduce_vertex('v3-z','v3+z')
```

```
# For blocks generation
def vnamegen(x0y0, x1y0, x1y1, x0y1):
return (x0y0+'-z', x1y0+'-z', x1y1+'-z', x0y1+'-z',
x0y0+'+z', x1y0+'+z', x1y1+'+z', x0y1+'+z')

b0 = bmd.add_hexblock(vnamegen('v0', 'v1', 'v8', 'v11'),
(Nx1, Ny1, 1),
'b0','fluid',
grading=SimpleGrading(1,1,1))

b1 = bmd.add_hexblock(vnamegen('v11', 'v8', 'v9', 'v10'),
(Nx1, Ny2, 1),
'b1','fluid',
grading=SimpleGrading(1,1,1))

b2 = bmd.add_hexblock(vnamegen('v1', 'v2', 'v7', 'v8'),
(Nx2, Ny1, 1),
'b2','fluid',
grading=SimpleGrading(1,1,1))

b3 = bmd.add_hexblock(vnamegen('v2', 'v3', 'v4', 'v7'),
(Nx3, Ny1, 1),
'b3','fluid',
grading=SimpleGrading(1,1,1))

b4 = bmd.add_hexblock(vnamegen('v7', 'v4', 'v5', 'v6'),
(Nx3, Ny2, 1),
'b4','fluid',
grading=SimpleGrading(1,1,1))


# For boundaries generation

bmd.add_boundary('wedge', 'front',
[
b0.face('t'),
b1.face('t'),
b2.face('t'),
b3.face('t'),
b4.face('t'),
]
)
bmd.add_boundary('wedge', 'back',
[
b0.face('b'),
b1.face('b'),
b2.face('b'),
```

```
b3.face('b'),
b4.face('b'),
]
)
bmd.add_boundary('wall', 'wall',
[
b1.face('n'),
b1.face('e'),
b2.face('n'),
b4.face('w'),
b4.face('n'),
]
)
bmd.add_boundary('patch', 'inlet',
[
b0.face('w'),
b1.face('w'),
]
)
bmd.add_boundary('patch', 'outlet',
[
b3.face('e'),
b4.face('e'),
]
)
bmd.add_boundary('empty', 'axis',
[
b0.face('s'),
b2.face('s'),
b3.face('s'),
]
)


# Vertex are labelled

bmd.assign_vertexid()


# And finally the dictionary is printed and saved to a file

print(bmd.format(),file=open('./system/blockMeshDict','w'))

print('blockMeshDict generated and copied in system folder')
```

Listing 2: Listing of the python script for the generation of the mesh

Now the mesh can be generated with this new `system/blockMeshDict`

```
foamJob -log-app blockMesh
```

and the mesh can be check with **paraFoam**.

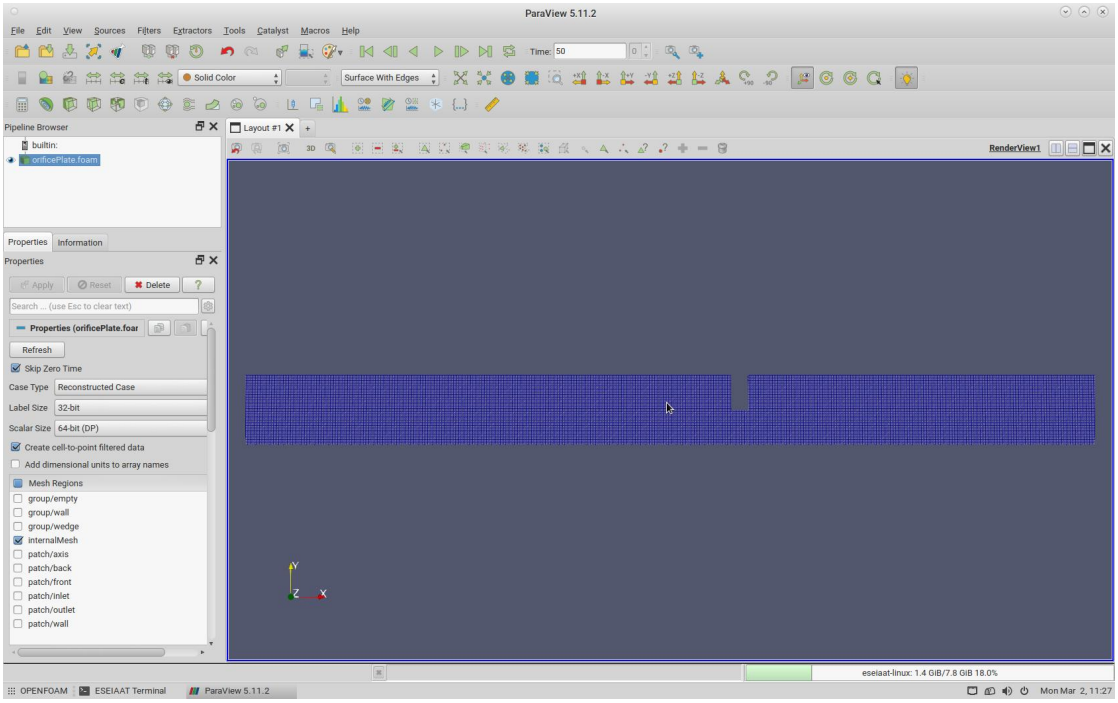

Figure 4: Orifice plate mesh

2.4 Boundary conditions and fluid properties

2.4.1 Velocity

Check the initial condition and boundary condition for velocity

```
cat 0/U
```

The output will be this:

```
...

Uinlet          (10 0 0);

dimensions      [0 1 -1 0 0 0 0];

internalField   uniform (0 0 0);

boundaryField
{
  inlet
```

```
    {
      type            fixedValue;
      value           uniform $Uinlet;
    }

    outlet
    {
      type            pressureInletOutletVelocity;
      value           uniform (0 0 0);
    }

    wall
    {
      type            noSlip;
    }

    #includeEtc "caseDicts/setConstraintTypes"
}
```

Some notes on this file:

1. The value of velocity is defined as a variable `Uinlet` at the beginning of the file and then it is used in the dictionary with `$Uinlet`.
2. 10 m/s is huge for our case of water. **Change it to** 1 m/s.
3. `pressureInletOutletVelocity` boundary condition for outlet is an special case where the pressure is specified. It is as zero-gradient fo outflow and with the velocity in the internal cell for inflow (reversed flow).
4. `#includeEtc "caseDicts/setConstraintTypes"` is a macro expansion that includes all the possible constraints. In this case it is useful because it avoids us to explicitly define the wedge patches.

2.4.2 Pressure

The pressure file, `0/p`, is OK: zero gradient in inlet and fixed value of $0\,m^2/s^2$ at the outlet

```
dimensions      [0 2 -2 0 0 0 0];

internalField   uniform 0;

boundaryField
{
inlet
{
type            zeroGradient;
```

```
}

outlet
{
type            fixedValue;
value           uniform 0;
}

wall
{
type            zeroGradient;
}

#includeEtc ''caseDicts/setConstraintTypes''
}
```

2.4.3 Turbulent variables

This simulation will be made with $k-\omega$ SST turbulence model. Two variables, k and ω have to be setup as boundary conditions in inlet, outlet and walls.

The `0/k` file has to be this one:

```
kInlet          0.00375;

dimensions      [0 2 -2 0 0 0 0];

internalField   uniform $kInlet;

boundaryField
{
inlet
{
type            turbulentIntensityKineticEnergyInlet;
intensity       0.05;
value           uniform \$kInlet;
}

outlet
{
type            inletOutlet;
inletValue      uniform \$kInlet;
value           uniform \$kInlet;
}

wall
{
type            kqRWallFunction;
```

```
  value           uniform \$kInlet;
  }

  #includeEtc ‘‘caseDicts/setConstraintTypes’’
  }
```

and this for 0/omega:

```
  omegaInlet      80.0;

  dimensions      [0 0 -1 0 0 0 0];

  internalField   uniform $omegaInlet;

  boundaryField
  {
    inlet
    {
      type            turbulentMixingLengthFrequencyInlet;
      mixingLength    0.0014;
      value           uniform $omegaInlet;
    }

    outlet
    {
      type            inletOutlet;
      inletValue      uniform $omegaInlet;
      value           uniform $omegaInlet;
    }

    wall
    {
      type            omegaWallFunction;
      value           uniform $omegaInlet;
    }

    #includeEtc "caseDicts/setConstraintTypes"
  }
```

According to this boundary conditions, values of k, kinetic turbulence energy, and ω, turbulent specific dissipation rate, are computed with

$$k = \frac{3}{2}\left(\|U\|I\right)^2 \tag{3}$$

and

$$\omega = \frac{k^{\frac{1}{2}}}{C_{\mu}^{\frac{1}{4}} L} \tag{4}$$

where $I = 0.05$ is the turbulence intensity, U is the average velocity of the flow, $C_\mu = 0.09$ and $L = 0.07D$, being D the diameter of the pipe, is the turbulence mixing length.

2.5 Viscosity of fluid

Since the fluid in this simulation will be water, we have to change the kinematic viscosity in `constant/transportProperties`

```
transportModel  Newtonian;

nu              [0 2 -1 0 0 0 0] 1e-06;
```

3 Running the simulation

3.1 Case preparation. Monitoring simulation

To monitor residuals and convergence of the simulation we are going to use the **`solverInfo`** function provided by openfoam.

Get this dictionary from the openfoam distribution

```
foamGetDictionary solverInfo
```

and edit it to include the variables k and ω.

```
#includeEtc "caseDicts/postProcessing/numerical/solverInfo.cfg"

fields (p U k omega);
```

Listing 3: `system/solverInfo` file.

Include this function in the `system/controlDict` file

```
...

functions
{
  #includeFunc    solverInfo
}
```

Listing 4: `fuctions` subdictionary of `system/controlDict` file.

By default, the simulation is set up to finish when residuals reach the value 10^4. We modify it to consider that it has converged fo 10^3, by editing the file `system/fvSolution`

```
...

SIMPLE
{
  residualControl
  {
    p               1e-3;
    U               1e-3;
    "(k|omega|epsilon)" 1e-3;
  }
  nNonOrthogonalCorrectors 0;
  pRefCell         0;
  pRefValue        0;

}

...
```

Listing 5: `SIMPLE` subdictionary of `system/fvSolution` file to control simulation convergence criteria.

3.2 Solver execution

```
foamJob -log-app simpleFoam
```

3.3 Monitoring convergence

We can plot the residuals with the command

```
foamMonitor -l postProcessing/solverInfo/0/solverInfo.dat
```

The option `-l` plots the vertical axis with logarithmic scale.

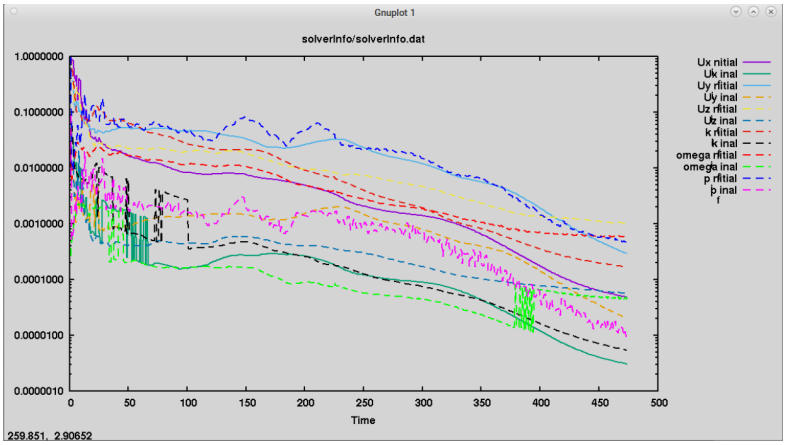

Figure 5: Residuals of simulation

4 Results and Validation

In paraview, with command `paraFoam`. visualize the last time step (converged solution)

4.1 Pressure field visualization

The expansion for the whole pipe diameter can be done with `Filter` → `Alphabetical` → `Reflect` with respect to the plane `Y Min`. Note the pressure drop in the orifice plate

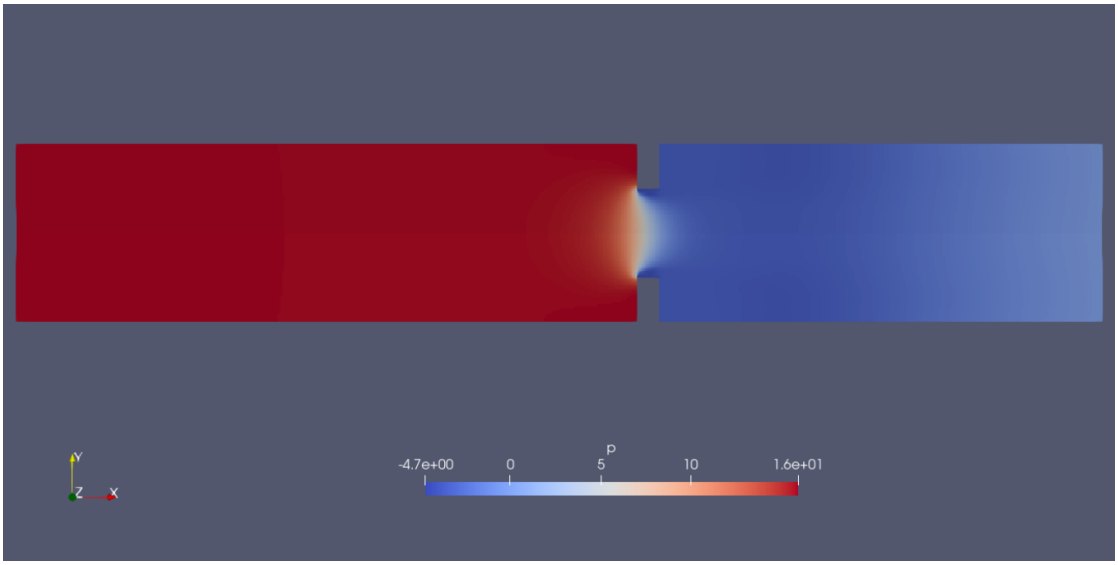

Figure 6: Pressure contours

4.2 Velocity field

The vena contracta is shown the velocity contoours.

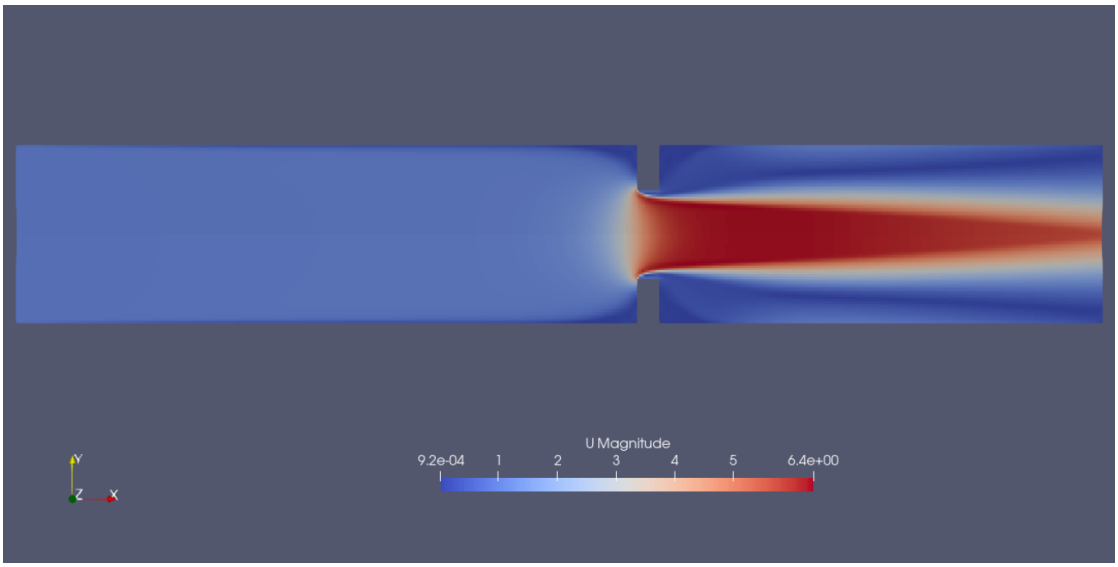

Figure 7: Velocity contour

The streamlines shown in Figure 8 are generated with filters:

1. `StreamTracer` with a vertical line located in $x = 0.09\,\mathrm{m}$ with a resolution of 50 points.
2. `Tube` filter with a radius of $0.2\,\mathrm{mm}$.

The reatachment point is very close to the outlet patch. That suggest that downstream region should be longer.

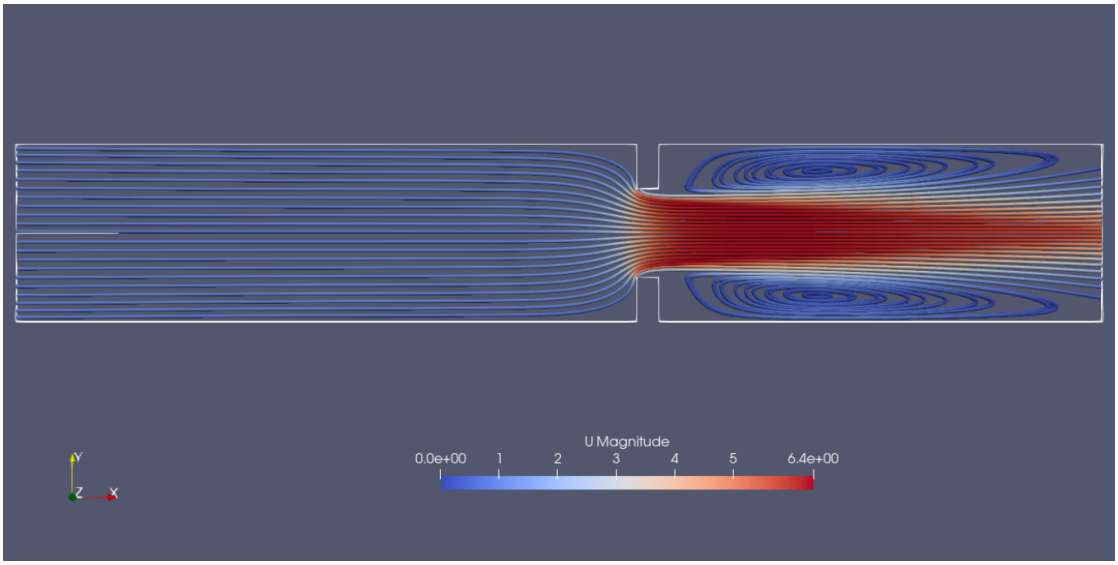

Figure 8: Streamlines of flow.

4.3 Coefficient of discharge calculation

Following equation (1), the discharge coefficient is computed as

$$C_d = \frac{Q\sqrt{1-\beta^4}}{A_o}\sqrt{\frac{\rho}{2\Delta p}} \tag{5}$$

to computationally compute Δp, let's plot the pressure distribution along the wall. We use the `Plot Over Line` with a line parallel to x axis, but located at $y = 0.0095\,\text{mm}$[1]. These data can be exported to a `csv` file with `File` $\rightarrow$ `Save Data` and processed with any program.

The upstream pressure has to be measured at a distance D of the orifice, that is, $x_1 = 50\,\text{mm}$, and downstream has to be at a distance $\frac{D}{2}$, $x_2 = 82.5\,\text{mm}$

[1] A line just in the wall, in $y = 0.01\,\text{m}$, wouldn't give any value because of the tolerance of the mesh. We measure the pressure 0.5 mm inside the domain.

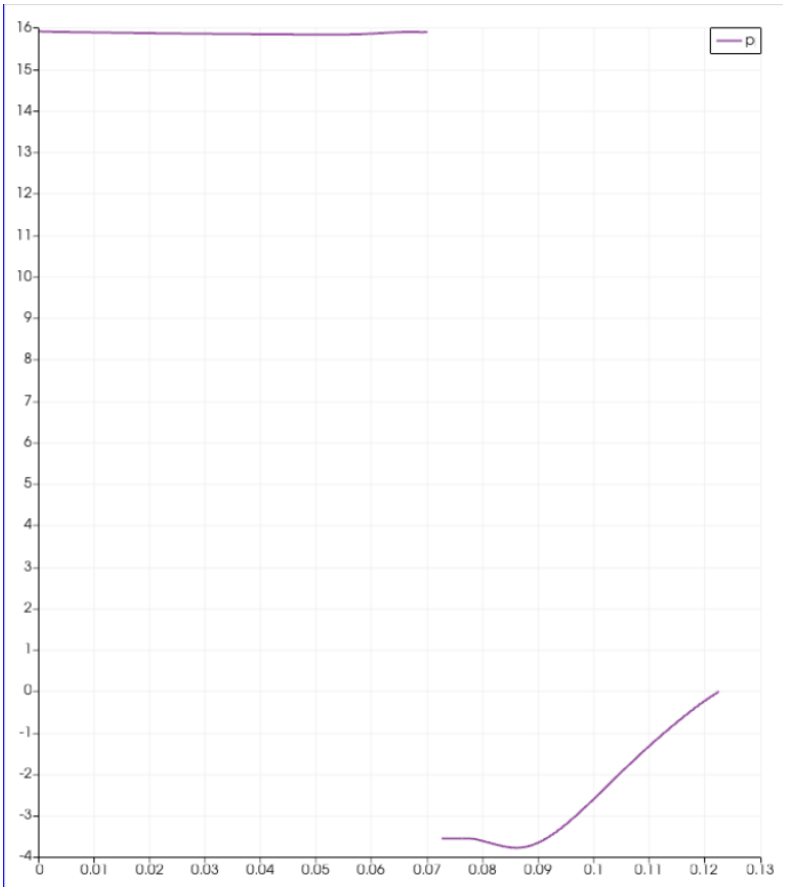

Figure 9: Pressure plot

This measurements can be processed with a jupyter notebook in order to compute C_d and validate it with the result from Reader-Harris-Gallagher equation. This can be done with the package `fluids`[3]

An example of such a jupyter notebook is show in Figure 10.

[1]:
```
import numpy as np
```

[2]:
```
U = 1 # m/s
D = 0.02  # m
d = 0.01  # m
mu = 0.001 # Pa·s
rho = 1000 # kg/m³
A = np.pi*D**2/4 # m²
Ao = np.pi*d**2/4 # m²
m = U*rho*A  # kg/s
ReD = U*rho*D/mu
print(f"mass flow rate = {m:.4f} kg/s")
print(f"Reynolds number = {ReD:.0f}")
```

```
mass flow rate = 0.3142 kg/s
Reynolds number = 20000
```

[3]:
```
beta = d/D
print(f"beta = {beta:.3f}")
```

```
beta = 0.500
```

[4]:
```
p1 = 15840
p2 = -3700
Delta_p = p1 - p2
```

[5]:
```
Cd_num = U*A*np.sqrt(1-beta**4)/Ao*np.sqrt(rho/(2*Delta_p))
print(f"Numerical value of Cd: {Cd_num:.3f}")
```

```
Numerical value of Cd: 0.620
```

[6]:
```
import fluids as fl
Cd = fl.flow_meter.C_Reader_Harris_Gallagher(m=m, rho=rho, mu=mu, D=D, Do=d, taps='D')
print(f"Experimental value of Cd: {Cd:.3f}")
```

```
Experimental value of Cd: 0.618
```

[7]:
```
error = np.abs(Cd_num-Cd)/Cd
print(f"error = {error*100:.2f} %")
```

```
error = 0.28 %
```

Figure 10: Jupyter Notebook for discharge coefficient computation and validation with experimental data.

5 Exercises

5.1 Modify beta ratio

Repeat the simulation with a different value of β by variation of the orifice diameter. It can be smaller than 0.5, $\beta = 0.4$, or bigger, $\beta = 0.6$, $\beta = 0.75$,...

Compare the outcome with the result from Reader-Harris-Gallagher equation.

5.2 Modify Reynolds

Repeat the simulation with a smaller value of Reynolds. It can be done either by modifying the inlet velocity, the pipe diameter or the viscosity. Remember to modify also turbulent variables according to this new configuration. Use "flanged taps" configuration for pressure measurement and compare with results from the publication by Manish et al.[4], where Re_D ranges from 4500 to 9000.

References

[1] Reader-Harris, M. J. (2015). *Orifice Plates and Venturi Tubes*. Springer International Publishing.

[2] ISO 5167-1:2022. *Measurement of fluid flow by means of pressure differential devices*. International Organization for Standardization, Geneva, Switzerland.

[3] Caleb Bell and Contributors (2016-2025). *fluids: Fluid dynamics component of Chemical Engineering Design Library (ChEDL)* https://github.com/CalebBell/fluids. https://fluids.readthedocs.io/fluids.flow_meter.html#orifice-plate-correlations

[4] Manish S. Shah, Jyeshtharaj B. Joshi, Avtar S. Kalsi, C.S.R. Prasad, Daya S. Shukla, *Analysis of flow through an orifice meter: CFD simulation*, Chemical Engineering Science, Volume 71, 2012, Pages 300-309, ISSN 0009-2509, https://doi.org/10.1016/j.ces.2011.11.022. https://www.sciencedirect.com/science/article/pii/S0009250911008219

Acknowledgments: The author acknowledges the assistance of DeepSeek, an AI assistant created by DeepSeek Company, for providing valuable support in structuring this document, formatting LaTeX code, and developing the tutorial content.

Disclaimer: All content has been reviewed, modified, and validated by the author. The author takes full responsibility for the accuracy, completeness, and educational quality of this material.